

Relatório das implementações das instruções do Risc-v

1. Inserção das Aritméticas de deslocamento(R-type):

1.1 Instruções XOR, SLL, SRL, SRA, SLT e SLTU :

Para essas instruções, foram feitas alterações dentro no módulo pl_alu, inserido mais configurações dentro do always combinacional referente as instruções em questão. Assim, estamos de fato dizendo como a ULA irá processar os dados que estão entrando. Lembrando que essas instruções possuem o mesmo opcode do tipo 7'b0110011. Então temos que adicioná-lo primeiro no modulo de control.

```
7 );
3
3 always_comb begin
3     case (Operation)
1         4'd01: ALUResult = $signed(SrcA) + $signed(SrcB);
2         4'd02: ALUResult = $signed(SrcA) - $signed(SrcB);
3         4'd04: ALUResult = SrcA | SrcB;
4         4'd05: ALUResult = SrcA & SrcB;
5         4'd06: ALUResult = SrcA ^ SrcB; // inserindo a xor
3         4'd07: ALUResult = SrcA << SrcB[4:0]; // utilizo apenas os 5 bits menos significativos de b, pois consigo representar a qtd maximo de deslocamento possivel no primeiro operando( 32 bits -- 25)
7         4'd08: ALUResult = SrcA >> SrcB[4:0]; // mesma logica do shift left
3         4'd09: ALUResult = $signed(SrcA) >>> SrcB[4:0]; // $signed para tratar o sinal e >>> representa o shift aritmetico(faz a copia do bit de sinal)
3         4'd11: ALUResult = 32'($signed(SrcA) < $signed(SrcB));
3         4'd12: ALUResult = 32'(SrcA < SrcB); // 32 para estender o resultado da comparacao(1 ou 0) para 32 bits e ficar coerente com a saida ALUResult
1
3
```

Dessa forma, dependendo da operação especificada, a ULA irá realizar uma dessas operações aritméticas. Um detalhe sobre esta é o uso do operador \$signed para levar em consideração o sinal do operando, diferenciando o shift lógico do shift aritmético.

Além disso, para definição desse operation, é necessário alterar o módulo alu_ctrl. Isso é necessário porque essa categoria de instruções aritméticas possuem o mesmo opcdoe e para diferenciá-las é necessário olhar os campos de funct3 e funct07. Dessa forma, a depender desses campos, nós definimos a saída operation que irá diretamente para o módulo da ALU defini qual operação será executada.

```

always_comb begin
    case (ALUOp)
        // Eu que defino o ALUOp

        2'b00: Operation = 4'd01; // se for 2'b00: categoria de acesso a memoria(alu forçada a fazer soma para o calculo do endereço)

        2'b01: Operation = 4'd02; // categoria de desvio condicional (alu forçada a fazer subtracao para comparar os valores dos registradores)

        2'b10: begin
            // categoria do tipo R-type (preciso olhar os funct agora)

            case (Funct3)
                3'h0: Operation = Funct7[5] ? 4'd02 : 4'd01; // SUB ou ADD (verifica o quinto bit do funct7)
                3'h6: Operation = 4'd04; // OR
                3'h7: Operation = 4'd05; // AND
                3'h2: Operation = 4'd11; // SLT
                3'h4: Operation = 4'd06; //XOR (funct 3 e 100)
                3'h1: Operation = 4'd07; //SLL
                3'h5: Operation = Funct7[5] ? 4'd09 : 4'd08; //SRL e SRA → mesmo funct 3 e os que diferencia é o quinto bit no funct 7
                3'h3: Operation = 4'd12; //SLTU
                default: Operation = 4'd01;
            endcase
        end
    end
end

```

Detalhe que as instruções SRL e SRA além de possuírem o mesmo opcode, também possuem o mesmo funct3. Assim, precisamos diferenciá-las pelo campo do funct7. Note que nesse caso, o quinto bit do funct 7 deles diferem. Logo ele vai selecionar a saída operation como sendo d09 ou d08 que correspondem ao código de sua respectiva operação.

2. Inserção das Aritméticas de deslocamento(I-type):

2.1 instruções ADDI, ANDI, ORI, SLTI, SLLI, SRLI, SRAI

Agora os opcode referentes a esse tipo de instrução é 7'b0010011, então primeiramente temos que inserir esse novo opcode no módulo de control.sv. Ainda dentro desse módulo, temos que definir os sinais de controle para as instruções do tipo I_type. Isso é importante pois cada componente do meu pipeline é controlado por sinais e existem operações que não vão utilizar determinado componente desse, como é o caso desse tipo de instrução que não acessa a memória por exemplo.

```

localparam R_TYPE = 7'b0110011;
localparam LOAD   = 7'b0000011;
localparam STORE  = 7'b0100011;
localparam BRANCH = 7'b1100011;
localparam I_TYPE = 7'b0010011; //

```

```
end
```

```
I_TYPE: begin // implementando os sinais de controle para as intrucoes do tipo I_type
    ALUSrc    = 1'b1;
    MemtoReg  = 1'b0;
    RegWrite  = 1'b1;
    MemRead   = 1'b0;
    MemWrite  = 1'b0;
    Branch    = 1'b0;
    ALUOp     = 2'b11;
end
```

```
/* Justificativa para os sinais para I_Type:
```

```
    ALUSrc    = 1'b0; → Como o segundo operando vem do extensor de imediato, esse sinal agora é 1(diferente da R_type)
    MemtoReg  = 1'b0; → Valor que sera escrito no reg_dest e o resultado da ALU(assim como em nas R_type)
    RegWrite  = 1'b1; → Sinal de escrita no registrador de destino = 1
    MemRead   = 1'b0; → I_type nao acessa a memoria de dados
    MemWrite  = 1'b0; → mesma justificativa da anterior
    Branch    = 1'b0; → Como I_type nao sao intrucoes de desvio, entao esse sinal e 0. (sempre PC + 4)
    ALUOp     = 2'b11. → Esse sinal sendo '11', irei definir uma nova categoria para tratar as I_type em ALU control*/
```

Após isso, é utilizada a mesma lógica para das instruções instruções sem immediatos para essa implementação no módulo alu_control. Afinal, a ULA não se importa se um tipo de instrução tem imediato e a outra não, pois ela é um bloco de hardware que apenas executas as instruções aritméticas. Assim é feito um reuso de hardware ao reaproveitar a lógica das instruções sem imediato, ou seja, seleciona o mesmo operation.

```
2'b11: begin // categoria i ty pe
    // nao sao todas de I_type que possuem funct 7
    case (Funct3) //
        3'h0: Operation = 4'd01; // ADDI → faz ADD (nao possui funct 7)
        3'h6: Operation = 4'd04; // ORI → faz uma or (nao possui funct 7)
        3'h7: Operation = 4'd05; // ANDi → sempre faz uma AND (nao possui funct 7)
        3'h2: Operation = 4'd11; // SLTi: Faz uma SLT (nao possui funct 7)
        3'h1: Operation = 4'd07; // SLLI → Faz SLL (possui funct 7)
        3'h5: Operation = Funct7[5] ? 4'd09 : 4'd08; // SRLI e SRAI diferenciam o funct 7 (possui funct 7)
```

```
    /*
    Esse campo trata dos immediatos do tipo I_type, mas essencialmente eu uso a mesma logica
    das minhas intrucoes do R_type. Afinal, a ULA em si só realiza a operacao, ela nao quer saber se uma
    das entradas veio de um extensor de imediato ou da memoria... Ela so realiza a operacao.
    Dessa forma, todas as intrucoes que encolvem imediato, essencialmente, realizam as mesmas
    contas na ula. E esse 2b11 é quem define de fato de onde ele esta vindo. o conteudo desse
    modulo, dessa forma, tem que ser o mesmo que a lógica da R_type.
    Dessa forma eu faco eu reuso de hardware.
    */
```

```
    default: Operation = 4'd01;
endcase
```

```
end
```

Por fim, é necessário alterar o módulo de extensão de sinal(sign_ext) para que possamos ampliar os 12 bits do campo de imediato para 32 bits, pois a ULA irá operar com entradas de 32 bits. Dessa forma, fazemos uma simples manipulação em system verilog:

```
`timescale 1ns / 1ps

module pl_sign_ext (
    input logic [31:0] Instr,
    output logic [31:0] ImmExt
);

    localparam LOAD = 7'b0000011;
    localparam STORE = 7'b0100011;
    localparam BRANCH = 7'b1100011;
    localparam I_TYPE = 7'b0010011; // para as instrucoes com imediato.

    always_comb begin
        case (Instr[6:0])
            LOAD: ImmExt = {{20{Instr[31]}}, Instr[31:20]};

            STORE: ImmExt = {{20{Instr[31]}}, Instr[31:25], Instr[11:7]};

            BRANCH: ImmExt = {{19{Instr[31]}}, Instr[31], Instr[7],
                               Instr[30:25], Instr[11:8], 1'b0};

            I_TYPE: ImmExt = {{20{Instr[31]}}, Instr[31:20]}; // logica semelhante com a do LOAD

            default: ImmExt = 32'b0;
        endcase
    end
endmodule
```
